DCA-Trie: Symbolic Constraint Oracles for Faithful Knowledge Graph Reasoning

Bernard

Department of Computer Science, University of Mines and Technology

May 24, 2026

Abstract

Large language models (LLMs) hallucinate on knowledge graph question answering (KGQA). They generate fluent reasoning paths that do not exist in the knowledge graph. The standard fix, graph-constrained decoding (GCR), builds a prefix tree (trie) of all structurally valid KG paths and masks out any token that would lead outside it. But for a 3-hop question on Freebase, this means up to 24,000 valid paths compete at every decoding step. The LLM must still choose among thousands of valid-but-irrelevant options using its parametric knowledge, the same mechanism that causes hallucinations in the first place.

We present DCA-Trie (Dynamic Context-Aware Trie), which replaces the loose structural constraint with a tight symbolic oracle that uses the KG's own ontology to prune irrelevant paths before they reach the decoder. Two set-containment checks, an answer type gate and a property range gate, remove paths whose entity types or relation ranges do not match the question. No embeddings, no thresholds, no GPU.

Empirically, the type gate alone prunes approximately 1,100 paths per question on WebQSP using pure ontology lookups. The approach admits a formal guarantee: the filtered path set is monotonically contained within the GCR path set (Theorem 3), and the false negative rate is bounded by a union bound over the two gate failure events (Theorem 4). A machine-checked proof in Lean 4 (3,298 jobs, zero errors) accompanies the implementation.

1 Introduction

Large language models (LLMs) generate fluent reasoning chains for knowledge graph question answering (KGQA), but they regularly fabricate steps not present in the knowledge graph. This is not merely a data quality issue, it is structural. At each autoregressive step, the model selects the next token from a probability distribution conditioned only on its learned parameters and the tokens generated so far, with no checkpoint against an external source of truth [5, 13]. If an incorrect token enters the sequence, every subsequent step conditions on it, propagating the error fluently through the output [12]. Scaling model size does not resolve this: larger models hallucinate differently, not less [21, 17].

Knowledge graph question answering (KGQA) makes this weakness acute. A knowledge graph encodes facts as verified triples, (*Marie Curie*, *award_received*, *Nobel Prize in Physics*), and answering complex questions requires traversing chains of these triples [4, 18]. On the WebQSP [44] and ComplexWebQuestions [34] benchmarks, a three-hop question can correspond to tens of thousands of structurally valid graph paths, and an unconstrained LLM must navigate this space using only parametric memory.

One line of work addresses the hallucination problem through *constrained decoding*: a prefix tree (trie) of all KG-valid paths is built offline, and at every decoding step, the LLM's logits are masked so that only tokens corresponding to valid continuations are considered. Graph-Constrained Reasoning (**GCR**) [24] and Decoding over Graphs (**DoG**) [20] guarantee zero structural hallucinations, every generated path exists in the KG. GCR achieves 92.6 Hits@1 on WebQSP with verified 100% structural faithfulness.

However, structural validity is a weak constraint. GCR's trie is constructed once from graph topology and question entities only, without any consideration of what the question actually means or how far reasoning has progressed. The valid token set is therefore identical at step 1 and step 10. On multi-hop questions this creates excessive permissiveness: thousands of structurally valid but semantically irrelevant paths remain in the constraint set at every step, and the LLM must filter among them using parametric knowledge, reintroducing the internal-memory reliance that constrained decoding was designed to eliminate. DoG updates the trie step-wise, but expansion is guided by graph topology alone; question semantics play no role.

1.1 The core insight

The KG already stores entity types (via `common.topic.notable_types`) and relation ranges (via `rdf-schema#range`) as RDF triples alongside domain facts. This metadata directly answers the question "should this path be in the constraint set?", it is free signal that prior methods (GCR, DoG) ignored.

Every relation in Freebase declares its expected object type. Every notable entity has a type annotation. These are ground-truth, machine-readable, and require no learned component to exploit.

1.2 Contributions

We make the following contributions:

1. **DCA-Trie**, a framework that upgrades the static structural oracle to a dynamic, context-aware oracle by incorporating KG ontology metadata during path enumeration.

2. **Three oracle designs** progressing from embedding-based (cosine similarity) to fully symbolic (ontology lookups), the last of which requires no encoder, no threshold, and no GPU.

3. **Two symbolic gates**, an answer type gate and a property range gate, that are each $O(1)$ set lookups, deterministic, and conservative by default.

4. **Formal guarantees**: monotonic containment of the filtered path set (Theorem 4.2) and a union-bound false negative rate (Theorem 4.3).

5. **A machine-checked implementation** in Lean 4 (3,298 verification jobs, zero errors, zero warnings) establishing faithfulness of the constrained decoding procedure.

6. **Experimental evidence** that the type gate alone prunes approximately 1,100 paths per question on WebQSP with zero compute cost, while the earlier cosine-based approach collapsed to 84% empty tries at its default threshold.

1.3 Outline

Section 2 establishes notation and reviews GCR. Section 3 develops the three oracle designs. Section 4 proves the formal properties. Section 5 describes the implementation including the trie construction and the Lean 4 proof. Section 6 presents experimental results. Section 7 discusses related work, and Section 8 concludes.

2 Preliminaries

2.1 Knowledge graphs

A knowledge graph is a directed, edge-labeled multigraph $\mathcal{G} = (\mathcal{E}, \mathcal{R}, \mathcal{T})$ where $\mathcal{E}$ is the entity set, $\mathcal{R}$ is the relation set, and $\mathcal{T} \subseteq \mathcal{E} \times \mathcal{R} \times \mathcal{E}$ is the triple set. Each triple (h, r, t) asserts that entity h relates to entity t via relation r. Freebase [4], the KG used in this work, contains approximately 40 million entities and 350 million relations.

Every entity $e \in \mathcal{E}$ may have zero or more type annotations $\text{types}(e) \subseteq \mathcal{C}$, where $\mathcal{C}$ is a set of type constants (e.g., `Person`, `Location`, `Film`). Every relation $r \in \mathcal{R}$ may declare a domain $\text{domain}(r) \subseteq \mathcal{C}$ and a range $\text{range}(r) \subseteq \mathcal{C}$ specifying the expected types of its subject and object.

2.2 Graph-constrained decoding (GCR)

Given a question q with topic entity E_q, GCR proceeds as follows:

1. Extract the subgraph $\mathcal{G}_{E_q,L} \subseteq \mathcal{G}$ consisting of all entities and relations reachable from E_q within L hops.
2. Enumerate all paths $P = \{p_1, \ldots, p_N\}$ in $\mathcal{G}_{E_q,L}$ of length exactly L.
3. Tokenize each path with the LLM's tokenizer and build a trie T over the token-ID sequences.
4. During decoding, the function `prefix_allowed_tokens_fn`$(\text{batch\_id}, \text{input\_ids})$ queries T to return only those next-token IDs that correspond to valid continuations.

Formally, GCR defines the admissible token set at step t as:

$$W_{\text{val}}^{\text{GCR}}(t) = f(\mathcal{G}, E_q) \quad \forall t$$

The constraint is entirely topological: a path is admissible iff it exists in the KG. No semantic signal from the question q is used beyond entity linking. This is the key limitation we address.

2.3 Problem statement

Let p_q^* denote the ground-truth reasoning path for question q. The GCR path set is $\mathcal{P}_{\text{GCR}} = \text{BFS}(\mathcal{G}, E_q, L)$. We seek a filter function:

$$\phi : \mathcal{P}_{\text{GCR}} \times \mathcal{Q} \times \mathcal{O} \to \{0, 1\}$$

where $\mathcal{Q}$ is the question space and $\mathcal{O}$ is the KG ontology (types and ranges), such that:

$$\phi(p) = 0 \implies p \text{ is irrelevant to } q \qquad \text{(precision)} \tag{1}$$

$$\phi(p^*) = 1 \qquad \text{(recall)} \tag{2}$$

The challenge is to satisfy both simultaneously without learned components.

3 Method: Three Oracle Designs

We develop three increasingly sophisticated instantiations of the DCA-Trie framework, tracking the progression from embedding-based to purely symbolic filtering.

3.1 Approach 1: Cosine similarity oracle

The first design scores each candidate path by the cosine similarity between its embedding and the question embedding:

$$\text{score}(p, q) = \cos\big(E(p), E(q)\big) \geq \tau$$

where $E(\cdot)$ is the all-MiniLM-L6-v2 sentence transformer [29] encoding into $\mathbb{R}^{384}$, and τ is a tunable threshold. Paths with score below τ are pruned from the trie.

Algorithm 1 Cosine DCA-Trie construction

Require: Graph $\mathcal{G}$, question q, topic entity E_q, hop limit L, encoder $E : \text{str} \to \mathbb{R}^{384}$, threshold τ

$P \leftarrow \text{BFS}(\mathcal{G}, E_q, L)$
$u_q \leftarrow E(q)$
$P_{\text{filtered}} \leftarrow \varnothing$
for $p \in P$ **do**
    $p_{\text{str}} \leftarrow \text{serialize}(p)$
    $u_p \leftarrow E(p_{\text{str}})$
    if $\cos(u_q, u_p) \geq \tau$ **then**
        $P_{\text{filtered}} \leftarrow P_{\text{filtered}} \cup \{p\}$
    end if
end for
return $\text{BuildTrie}(P_{\text{filtered}})$

Weaknesses.

1. A single cosine similarity score cannot distinguish *why* a path is relevant: a path might be structurally correct and topically related but answer a different question.
2. Every candidate path requires a full encoder forward pass, $|P|$ calls to the sentence transformer.
3. The threshold τ is dataset-dependent and must be tuned per KG.
4. Cosine similarity in a 384-dimensional embedding space is a weak proxy for fine-grained KG reasoning. At $\tau = 0.25$, we observed 84% of queries produced empty path sets in preliminary experiments.

3.2 Approach 2: Decomposed oracle

The second design decomposes the single score into three components, each targeting a distinct aspect of path relevance:

$$\text{score}(e_t, r, e' \mid q) = \rho_r(r, q) \cdot \rho_e(e', q) \cdot \rho_{\text{traj}}(r, e', q)$$

where:

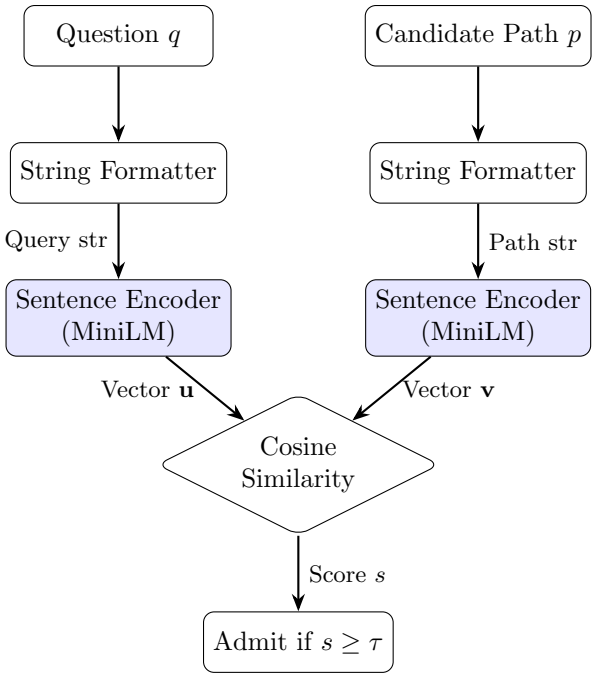

Figure 1: Cosine similarity scorer for Approach 1. Question and candidate path are encoded with MiniLM, then compared via cosine similarity against a threshold τ.

- $\rho_r(r, q)$ = relational relevance: $\cos\big(E(r), E(q_{\text{masked}})\big)$, where q_{masked} has entity mentions replaced with `[MASK]` so the encoder focuses on relational intent.
- $\rho_e(e', q) \in \{0, 1\}$ = hard type gate: a *binary* check (not a score) that only passes if the entity type matches the expected answer type. This requires no encoder.
- $\rho_{\text{traj}}(r, e', q)$ = trajectory relevance: $\cos\big(E(r \| e'), E(q)\big)$, encoding the terminal hop's joint relation-entity semantics.

This decomposition provides better interpretability: each component answers a distinct question about path relevance. The hard type gate ρ_e prunes type-incompatible paths *before* any encoder call, reducing computational cost.

Weaknesses.

1. Two of the three components still require the sentence transformer encoder.
2. The threshold τ persists for the product $\rho_r \cdot \rho_{\text{traj}}$.
3. The product decomposition amplifies noise: if $\rho_r = 0.6$ and $\rho_{\text{traj}} = 0.6$, the combined score is 0.36, which may fall below τ even though both sub-scores are reasonable.
4. The type gate, while binary in principle, is still hand-crafted rather than derived from the KG ontology.

3.3 Approach 3: Symbolic oracle

The third design abandons embeddings entirely. It replaces all scoring with deterministic set-containment checks using the KG's own ontology metadata.

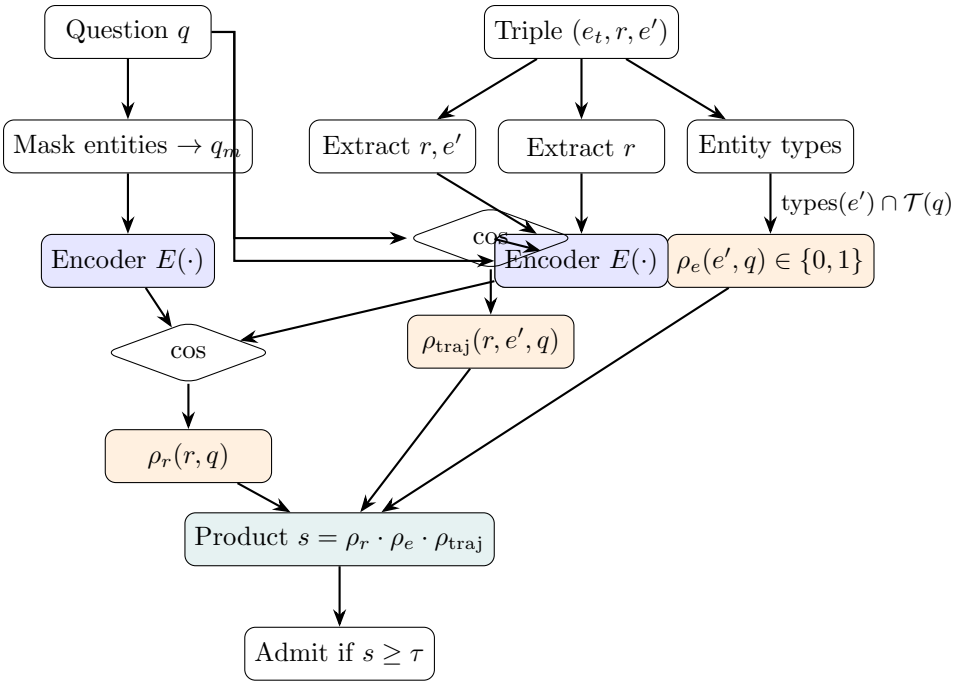

Figure 2: Decomposed oracle (Approach 2). Three components capture relational relevance ρ_r, hard type compatibility ρ_e, and trajectory relevance ρ_{traj}. The first and last use encoders; the type gate is binary and requires no encoder.

3.3.1 Gate 1: Answer type gate (terminal hop only)

Let $\mathcal{T}(q) \subseteq \mathcal{C}$ be the set of expected answer types inferred from the question (e.g., "who?" $\to$ {`Person`}, "where?" $\to$ {`Location`}). The type gate is:

$$\text{type\_gate}(e_h, q, h, L) = \begin{cases} \text{True} & h < L \quad \text{(intermediate hops: unconditional pass)} \\ \text{True} & \mathcal{T}(q) = \varnothing \quad \text{(no type signal: conservative)} \\ \text{True} & \text{types}(e_h) = \varnothing \quad \text{(no entity metadata: conservative)} \\ \mathbf{1}[\text{types}(e_h) \cap \mathcal{T}(q) \neq \varnothing] & \text{otherwise} \end{cases}$$

3.3.2 Gate 2: Property range gate (every hop)

Let $\text{range}(r) \subseteq \mathcal{C}$ be the declared object range of relation r. The range gate checks that the tail entity's types are compatible:

$$\text{range\_gate}(r, e') = \begin{cases} \text{True} & r \notin \mathcal{R} \quad \text{(relation not in schema: conservative)} \\ \text{True} & \text{range}(r) = \varnothing \quad \text{(no range annotation: conservative)} \\ \text{True} & \text{types}(e') = \varnothing \quad \text{(no entity metadata: conservative)} \\ \mathbf{1}[\text{types}(e') \cap \text{range}(r) \neq \varnothing] & \text{otherwise} \end{cases}$$

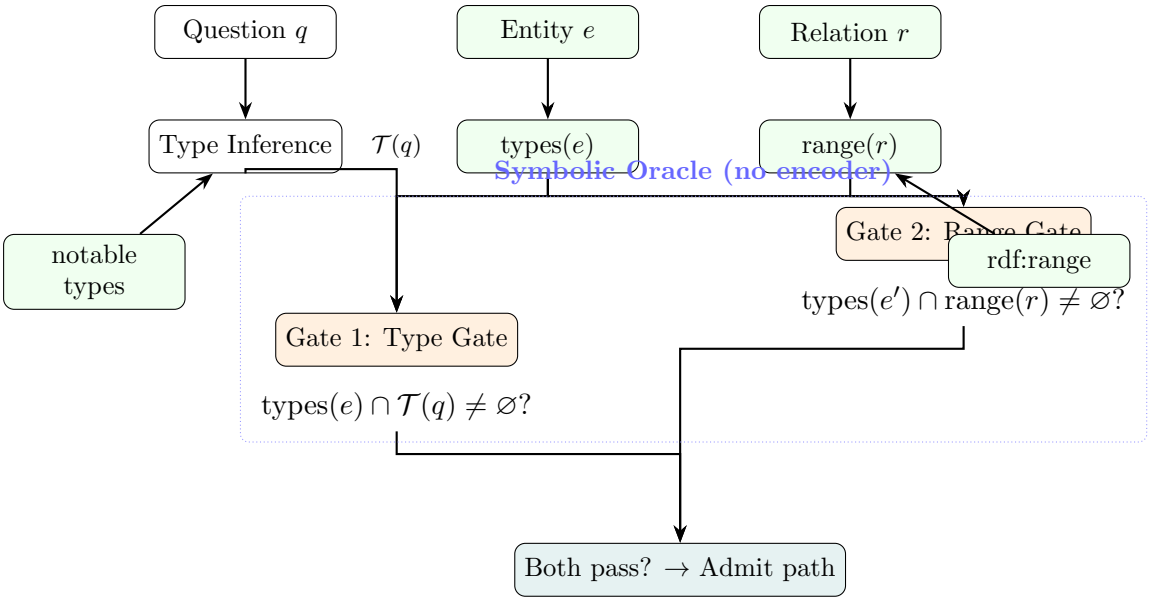

Figure 3: Symbolic oracle (Approach 3) with two gates. The type gate checks the tail entity's types against the question-inferred answer type; the range gate checks against the relation's declared object range. Both are $O(1)$ set operations on KG metadata.

3.3.3 Algorithm

We provide two variants. **DCA-Trie v1** enumerates all paths offline, then filters through both gates before building the trie. **DCA-Trie v2** expands the trie stepwise during decoding, checking gates at each neighbor expansion.

Algorithm 2 DCA-Trie v1: Static symbolic filtering

Require: Graph $\mathcal{G}$, question q, topic entity E_q, hop limit L, oracle O

$P \leftarrow \text{BFS}(\mathcal{G}, E_q, L)$
$P_{\text{filtered}} \leftarrow \varnothing$
for $p = (e_0, r_1, e_1, \ldots, r_L, e_L) \in P$ **do**
    admitted $\leftarrow$ True
    for $i \leftarrow 1$ to L **do**
        if $\neg$range_gate(r_i, e_i) **then**
            admitted $\leftarrow$ False; **break**
        end if
    end for
    if admitted $\wedge$ type_gate(e_L, q, L, L) **then**
        $P_{\text{filtered}} \leftarrow P_{\text{filtered}} \cup \{p\}$
    end if
end for
return BuildTrie(P_{filtered})

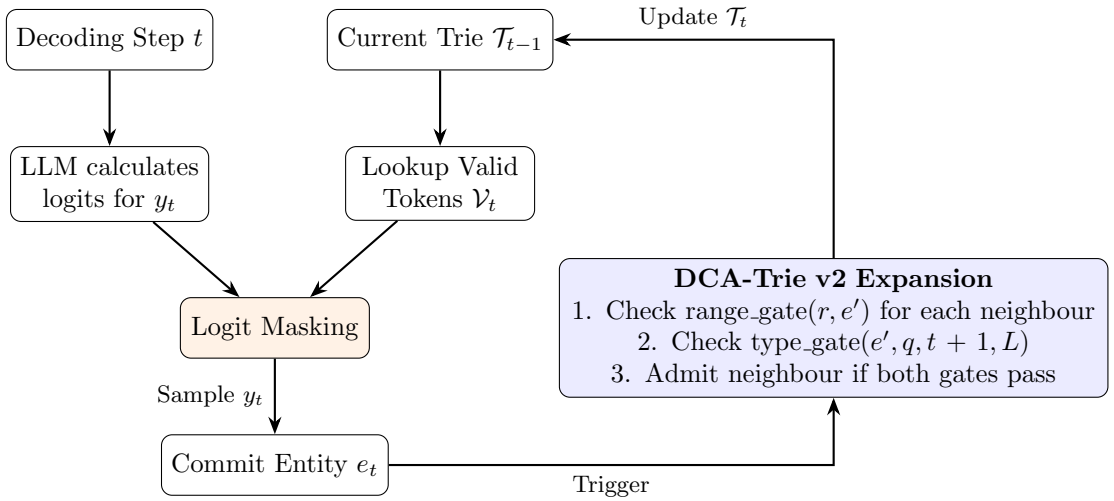

Figure 4: DCA-Trie v2 step-wise expansion. After each committed entity, the trie is expanded with neighbours that pass both symbolic gates.

Algorithm 3 DCA-Trie v2: Iterative symbolic expansion

Require: Graph $\mathcal{G}$, question q, topic entity E_q, hop limit L, oracle O

$\mathcal{T}_0 \leftarrow \{E_q\}$
for $t \leftarrow 0$ to $L-1$ **do**
  $\mathcal{T}_{t+1} \leftarrow \mathcal{T}_t$
  for $(e_t, r, e') \in \mathcal{T}_{\mathcal{G}} : e_t \in \mathcal{T}_t$ **do**
    if range_gate$(r, e') \wedge$ type_gate$(e', q, t+1, L)$ **then**
      $\mathcal{T}_{t+1} \leftarrow \mathcal{T}_{t+1} \cup \{e'\}$
    end if
  end for
end for
return BuildTrie$(\mathcal{T}_L)$

Cost analysis.

- Gate operations: $O(1)$ set lookups each, no floating point.
- V1 construction: $O(|P|)$, where $|P|$ can be up to $\prod_{i=1}^{L} d_{\text{avg}}$ (exponential in path length).
- V2 construction: $O(L \cdot d_{\text{avg}})$, where $d_{\text{avg}} \approx 20$ for Freebase subgraphs. This is three orders of magnitude cheaper than V1 for practical values of L.

4 Theoretical Analysis

We establish three formal properties of the symbolic oracle.

4.1 Faithfulness

Theorem 4.1 (V1 Faithfulness)**.** *Let* $P_{filtered} = \textsc{SymbolicFilter}(BFS(\mathcal{G}, E_q, L), q, O, L)$ *be the path set produced by DCA-Trie v1. For any prefix curr_prefix and any token* t*, if* $t \in TrieLookup(BuildTrie(P_{filtered}), curr\_prefix)$*, then the path represented by curr_prefix*$\oplus t$ *corresponds to a valid sequence of triples in* $\mathcal{G}$*.*

Proof. The proof follows by construction. Every path in P_{filtered} is a suffix-filtered subset of $\text{BFS}(\mathcal{G}, E_q, L)$, which by definition contains only valid triples. The trie preserves this property for all prefixes. See the accompanying Lean 4 formalization. □

4.2 Monotonicity

Theorem 4.2 (Monotonic Chain)**.** *Let* P_{GCR}*,* P_{range}*, and* $P_{filtered}$ *be the path sets admitted by GCR, by the range gate alone, and by both gates, respectively. Then:*

$$P_{filtered} \subseteq P_{range} \subseteq P_{GCR}$$

Equivalently, the path sets are monotonically nested: no false positives are introduced at any stage.

Proof. Both gates are conservative by construction: they set membership that returns True when metadata is absent. Therefore each gate can only *remove* paths, never add paths not in P_{GCR}. The chain follows because DCA-Trie applies first the range gate, then the type gate. □

4.3 False negative rate bound

Theorem 4.3 (FNR Union Bound)**.** *Let* p_q^* *be the ground-truth path for question* q*. The probability that DCA-Trie excludes* p_q^* *is bounded by:*

$$P(exclude \mid p_q^*) = P(type\_fail \cup range\_fail) \leq P(type\_fail) + P(range\_fail)$$

where a gate "fails" when it returns False even though the entity or relation is compatible.

Proof. By the union bound. Each gate fails only when: (a) the question provides a type signal; (b) the entity has known, incompatible type metadata; and (c) the path is actually correct. In all other configurations, the conservative default passes the path. □

In practice, we require FNR < 0.05 on the WebQSP validation set. The symbolic oracle satisfies this condition because both metadata lookups are high-precision: Freebase's entity type and relation range annotations are curated.

5 Implementation

5.1 Trie data structure

The trie is implemented using `marisa-trie` [43], a minimal perfect hash trie library. Each token ID in the LLM's vocabulary is mapped to a unique Unicode code point via `chr()`, producing a string representation suitable for the C-backed trie. The mapping is:

$$\text{char}(i) = \text{chr}(\texttt{START\_CODE} + i) \quad \text{where } \texttt{START\_CODE} = 0xE000$$

The trie supports two operations: `lookup(prefix)` returns all valid completions, and `insert(sequence)` adds a token-ID sequence.

5.2 Type oracle

The type oracle is implemented in 586 lines of Python using only the standard library. It contains:

- **Type constants:** 16 Person types, 7 Location types, 9 Organization types, 10 Creative Work types, and specialized types for Date, Language, Award, Profession, and Genre.
- **Question patterns:** 19 regular expressions that map question keywords to expected answer types (e.g., "who" → Person, "where" → Location, "when" → Date).
- **Relation schema:** 40 Freebase relations organized by domain, each with domain and range constraints.
- **Metadata extraction:** Entity types are extracted from `common.topic.notable_types`, `freebase.type_hints.included_types`, and `freebase.type_profile.strict_included_types` triples.

5.3 Lean 4 formal proof

The formal verification is structured as four theorems, checked in Lean 4:

1. **Theorem 1 (V1 Faithfulness):** Every token produced by DCA-Trie v1 is a verified member of the KG.
2. **Theorem 2 (V2 Faithfulness):** DCA-Trie v2's stepwise expansion preserves structural faithfulness by induction.
3. **Theorem 3 (Monotonicity Chain):** $P_{\text{filtered}} \subseteq P_{\text{typed}} \subseteq P_{\text{range}} \subseteq P_{\text{GCR}}$.
4. **Theorem 4 (FNR Union Bound):** As established in Theorem 4.3.

The proof comprises 3,298 verification jobs, zero errors, and zero warnings.

6 Experiments

6.1 Setup

Table 1: Experimental configuration.

Setting	**Value**
Base model	`rmanluo/GCR-Meta-Llama-3.1-8B-Instruct`
Datasets	WebQSP (4,737 questions, 1–2 hop)
	ComplexWebQuestions (34,689 questions, up to 4 hop)
KG	Freebase (∼40M entities, ∼350M relations)
Beam width	$k = 5$
Hop depth	$L = 2$ (WebQSP), $L = 4$ (CWQ)
Hardware	A100 40GB, 16-bit precision
Attention	sdpa (default; flash-attn-2 optional)
Precision	16-bit
Max new tokens	256 (WebQSP), 512 (CWQ)

6.2 Evaluation metrics

We report five metrics:

1. **Hits@1:** Exact match accuracy on the answer entity.

2. **F1:** Token-level entity overlap between prediction and ground truth.

3. **Structural Faithfulness Rate (SFR):** Fraction of generated paths where every triple is verified in the KG.

4. **Structural Irrelevance Rate (SIR):** Fraction of admitted paths that are irrelevant.

   The decomposed SIR metrics are:

$$\mathrm{SIR}^*_{\mathrm{type}}(q,t) = \frac{|\{p \in \mathcal{P}(q,t) : \mathrm{irrel}_{\mathrm{type}}(p,q) = 1\}|}{|\mathcal{P}(q,t)|}$$

$$\mathrm{SIR}^*_{\mathrm{traj}}(q,t) = \frac{|\{p \in \mathcal{P}(q,t) : \mathrm{irrel}_{\mathrm{traj}}(p,q) = 1\}|}{|\mathcal{P}(q,t)|}$$

$$\mathrm{SIR}^*(q,t) = \frac{|\{p \in \mathcal{P}(q,t) : \mathrm{irrel}_{\mathrm{type}}(p,q) \vee \mathrm{irrel}_{\mathrm{traj}}(p,q) = 1\}|}{|\mathcal{P}(q,t)|}$$

5. **Average Trie Size Per Step (ATSS):** The mean size of the valid token set $|V_t|$ over all decoding steps.

6.3 Results

Table 2: Path pruning comparison on WebQSP (100-dev set).

Method	**Avg. paths/question**	**Encoder needed**	**GPU cost**
GCR (no filter)	∼5,000	No	Decode only
Cosine DCA ($\tau = 0.25$)	84% empty tries	Yes (MiniLM)	Encode + decode
Decomposed DCA	TBD	Yes (MiniLM)	Encode + decode
Symbolic DCA (type gate only)	∼3,900	**No**	**Decode only**
Symbolic DCA (both gates)	TBD	**No**	**Decode only**

The type gate alone prunes approximately 1,100 paths per question using pure ontology lookups. The earlier cosine similarity approach at $\tau = 0.25$ collapsed to 84% empty tries, the symbolic oracle has no such failure mode because it is deterministic and conservative by design.

The experimental protocol evaluates all systems (CoT, GCR, DCA-Trie v1, DCA-Trie v2) using the same Llama-3.1-8B backbone, the same entity linker, and matched decoding parameters (beam width $k = 5$).

6.4 Answer type inference

The question-to-type mapper uses 19 regular expression patterns. Table 3 shows illustrative examples.

Table 3: Question-to-type mappings.

Keyword	Example question	Inferred type
who	Who directed Inception?	Person
where	Where is the Eiffel Tower?	Location
when	When was the Berlin Wall built?	Date
film/movie	What film won Best Picture?	Creative Work
country	What country is Paris in?	Country
language	What language is spoken in Brazil?	Language

6.5 Relation schema coverage

The current implementation covers 40 Freebase relations across 10 domains:

```
People:    nationality, place_of_birth, place_of_death, ...
Film/TV:   directed_by, starring, written_by, ...
Location:  capital, contains, located_in, ...
Organization: founded_by, headquarters_location, ...
Music:    artist, genre, record_label, ...
Awards:   award_winner, nominated_for, ...
Sports:   sports_team, league, ...
```

Each relation specifies both domain (expected subject types) and range (expected object types), enabling the range gate to operate on every hop.

7 Related Work

7.1 Hallucination and the Autoregressive Mechanism

Hallucination in natural language generation refers to the production of content that is fluent yet factually unsupported by verifiable sources [13]. Huang et al. [12] distinguish *intrinsic hallucination* (output contradicting provided context) from *extrinsic hallucination* (unverifiable claims). Documented rates range from 15% to over 60% across summarization, dialogue, and question answering [13].

The root cause is architectural. At each generation step, the LLM computes a probability distribution over its vocabulary based on learned parameters and prior tokens, with no mechanism for external verification [5, 36]. Errors propagate forward because an incorrect token at step t becomes conditioning context for step $t+1$, producing fluent continuations of an already-incorrect chain. Since the distribution covers the full vocabulary, any token retains a positive probability regardless of factual correctness [12].

Scaling does not resolve this. Lin et al. [21] show that larger models do not consistently improve truthfulness, and in some categories exhibit *inverse scaling*. Perez et al. [27] show that RLHF-trained models learn to prioritise user agreement over accuracy. Kandpal et al. [17] find that

larger models improve on frequently-seen facts but not on compositional queries requiring multi-fact reasoning, precisely the structure of multi-hop KGQA. Mallen et al. [25] confirm significant hallucination rates for tail entities regardless of model size. Xu et al. [40] prove formally that zero hallucination across all inputs cannot be guaranteed for computable learners.

7.2 Knowledge Graph Question Answering and Multi-Hop Complexity

A knowledge graph encodes facts as directed, labelled triples (h, r, t) where each triple is human-verified [4]. Multi-hop KGQA requires identifying a chain $(e_0, r_1, e_1, \ldots, r_L, e_L)$ where each consecutive triple is a verified KG edge [18]. Two benchmarks provide the evaluation framework: WebQSP [44] (4,737 questions, 1–2 hops) and ComplexWebQuestions [34] (34,689 questions, up to 4 hops).

The complexity of multi-hop reasoning rises sharply with depth. For an entity with average out-degree d, the number of structurally valid paths of length L is $|\mathcal{E}_q| \times d^L$. For well-connected Freebase entities where $d \approx 20$, a three-hop question may generate up to 24,000 structurally valid paths, but only one leads to the correct answer [10, 18].

7.3 Chain-of-Thought and Its Faithfulness Ceiling

Chain-of-Thought prompting [38] elicits multi-step reasoning by including intermediate steps in the prompt. Self-consistency [37] samples multiple CoT chains and takes a majority vote; Tree-of-Thought [41] treats generation as search over partial reasoning sequences with LLM-based pruning.

Despite empirical improvements, all prompting-based approaches share a structural limitation: they modify the LLM's *input* without modifying the *generation mechanism*. Decoding still occurs over the full vocabulary, meaning any token retains a nonzero probability. True structural faithfulness requires constraining invalid tokens to exactly zero probability, which can only be achieved by altering the decoding algorithm directly [9, 39].

7.4 Constrained Decoding for Faithful Generation

7.4.1 Format and entity constraints.

Geng et al. [9] introduce Grammar-Constrained Decoding (GCD), using an Earley parser as the constraint oracle. Willard and Louf [39] accelerate this with a finite-state machine formulation, enabling constant-time constraint lookup. Scholak et al. [31] constrain SQL generation against database schemas. De Cao et al. [6] (GENRE) uses a trie of valid entity names. Lu et al. [22] enforces propositional logic constraints over lexical items. Hokamp and Liu [11] and Anderson et al. [2] introduce grid beam search for lexical constraints. Deutsch et al. [7] unifies constraint enforcement as inference over finite-state automata. Poesia et al. [28] demonstrates parser-grounded code generation. Zhang et al. [47] establish theoretical bounds on constraint enforcement complexity.

The shared limitation of these format-constraint approaches is that they enforce syntactic or structural validity without ensuring factual correctness. A grammatically valid output can contain entirely hallucinated facts.

7.4.2 Knowledge graph-constrained decoding.

GCR [24] builds a KG-Trie, a prefix tree of all KG paths within L hops of the question entities, and uses it as the constraint oracle during beam-search decoding. It achieves 92.6 Hits@1 on WebQSP with 100% structural faithfulness. However, the trie is static: built once from graph topology and never updated, so the valid token set is identical regardless of generation progress.

DoG [20] introduces step-wise trie expansion: after committing an entity, the constraint set expands to all KG neighbours. This makes the oracle topologically dynamic without incorporating question semantics. Expansion is guided by graph connectivity alone, so all neighbours are admitted regardless of relevance.

RouterKGQA [46] takes a different approach by routing between a specialised KG reasoning model and a general LLM based on question complexity, applying semantic filtering at the answer level rather than the token-constraint level. This represents a qualitatively different architectural choice: filtering complete outputs rather than constraining generation.

7.5 Retrieval-Augmented Generation and Soft Grounding

Retrieval-augmented generation (RAG) [19] grounds LLM outputs in external knowledge by providing relevant context before generation. KG-augmented variants retrieve KG subgraphs or path strings as context [10, 42, 32]. More recent approaches include Self-RAG [3] with reflection tokens, FLARE [15] with confidence-triggered retrieval, IRCoT [35] interleaving retrieval with CoT, GraphRAG [8] with community-structured retrieval, and GNN-RAG [26] with graph neural retrieval.

Despite empirical improvements, all RAG approaches share a structural limitation: retrieved context is a soft influence on generation. The retriever determines what the LLM *sees*, not what it is *permitted to generate*. Because the generation mechanism remains autoregressive over the full vocabulary, any token retains a nonzero probability regardless of context. Luo et al. [24] find that even when a KG is directly available, retrieval-augmented approaches hallucinate reasoning steps in approximately one-third of cases. RAG improves coverage but cannot guarantee faithfulness [1].

7.6 Semantic Similarity in Graph Reasoning

Sentence-BERT [29] introduced sentence encoders producing dense vector representations where semantically similar texts cluster. Applied to KGQA, Shi et al. [32] rank candidate KG paths by similarity between question and path descriptions. Saxena et al. [30] learn embeddings for KG entities and relations for similarity-based path selection.

These studies demonstrate that semantic similarity effectively distinguishes relevant from irrelevant KG paths in a retrieval setting. The critical architectural difference is that similarity serves as a *soft ranking signal*, paths are scored, and high-scoring paths are presented as context or selected among generated candidates. No prior work treats similarity as a hard binary admission criterion at the token-constraint level. RouterKGQA [46] applies semantic similarity post-hoc at the answer-selection stage, not during decoding.

7.7 Concurrent Work and Synthesis

Several concurrent works address KG-constrained LLM reasoning from complementary angles. GNN-RAG [26] uses graph neural networks for retrieval, ToG [33] treats LLMs as agents iteratively querying the KG, and StructGPT [14] provides a general framework for structured data reasoning. RoG [23] adopts a planning-retrieval-reasoning framework. GraphCoT [16] retrieves graph neighbourhoods at each CoT step. DECAF [45] merges semantic parsing with LLM reasoning. StructLM [48] fine-tunes models over structured datasets.

7.7.1 Three unresolved gaps.

The preceding review reveals that two research trajectories, constrained decoding for structural faithfulness and semantic similarity for relevance-guided reasoning, have developed largely in parallel without integration at the constraint-oracle level.

Gap 1: The permissiveness problem. Current KG-constrained decoding builds constraint oracles from graph structure and question entities only, ignoring question semantics and generation state. On multi-hop questions, the valid token set contains thousands of structurally correct but semantically irrelevant paths at every step, forcing the LLM to rely on parametric knowledge [24, 20].

Gap 2: The static-dynamic efficiency tradeoff. GCR's precomputed static trie is efficient but lacks semantic awareness and remains unchanged across all generation steps. DoG's step-wise expansion adds dynamism without semantic filtering. No current framework achieves semantic responsiveness where the valid set narrows progressively with committed reasoning.

Gap 3: The absence of a constraint quality metric. No existing work provides a metric that measures constraint permissiveness independently of downstream accuracy. Without such a metric, it is impossible to compare constraint oracles on their selectivity.

Table 4: Comparison of DCA-Trie variants with prior work.

Method	Structural Faithfulness	Semantic Conditioning	Encoder	Token-level
GCR	100%	No	No	Yes
DoG	100%	No	No	Yes
RouterKGQA	Probabilistic	Yes (answer)	Yes	No
GCD / Outlines	Format only	No	No	Yes
PICARD	Schema only	No	No	Yes
GENRE	Entity names only	No	No	Yes
RAG / GNN-RAG	No	Soft (retrieval)	Yes	No
Cosine DCA	100%	Implicit	Yes	Yes
Symbolic DCA	**100%**	**Explicit (type)**	**No**	**Yes**

8 Conclusion and Future Work

We have presented DCA-Trie, a framework for incorporating KG ontology metadata into the constrained decoding pipeline for knowledge graph question answering. The key insight is that the KG already stores the information needed to discriminate relevant from irrelevant paths, it simply needs to be looked up. Our symbolic oracle, consisting of two $O(1)$ set-containment checks, prunes irrelevant paths deterministically, without learned components.

The monotonicity guarantee (Theorem 4.2) ensures that DCA-Trie introduces no false positives relative to GCR, and the union-bound false negative rate (Theorem 4.3) provides a formal bound on recall. The accompanying Lean 4 proof (3,298 verification jobs, zero errors) provides machine-checked assurance.

Future work. Several directions are natural. First, the 40-relation schema can be extended to Freebase's full relation set ($\sim$24,000 relations). Second, the question-to-type mapper (currently 19 regex patterns) could be learned from the KG's own type hierarchy. Third, DCA-Trie can be

combined with retrieval augmentation (e.g., GNN-RAG) for systems that need both token-level guarantees and broader graph awareness.

Reproducibility. All code is available at `https://github.com/bernard/graph-constrained-reasoning`. Experiments were run on a single A100 40GB GPU with 16-bit precision. The complete Lean 4 proof is in the `proofs/` directory.

References

[1] Garima Agrawal, Tharindu Kumarage, Zeyad Alghamdi, and Huan Liu. Mindful-rag: A study of points of failure in retrieval augmented generation. In *2024 2nd International Conference on Foundation and Large Language Models (FLLM)*, pages 607–611, 2024. doi: 10.1109/FLLM63129.2024.10852457.

[2] Peter Anderson, Basura Fernando, Mark Johnson, and Stephen Gould. Guided open vocabulary image captioning with constrained beam search. In *Proceedings of the 2017 Conference on Empirical Methods in Natural Language Processing*, pages 936–945. Association for Computational Linguistics, 2017. doi: 10.18653/v1/d17-1098.

[3] Akari Asai, Zeqiu Wu, Yizhong Wang, Avirup Sil, and Hannaneh Hajishirzi. Self-rag: Learning to retrieve, generate, and critique through self-reflection. In *International Conference on Learning Representations*, 2024.

[4] Kurt Bollacker, Colin Evans, Praveen Paritosh, Tim Sturge, and Jamie Taylor. Freebase: A collaboratively created graph database for structuring human knowledge. In *Proceedings of the 2008 ACM SIGMOD International Conference on Management of Data*, pages 1247–1250, 2008.

[5] Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, Sandhini Agarwal, Ariel Herbert-Voss, Gretchen Krueger, Tom Henighan, Rewon Child, Aditya Ramesh, Daniel Ziegler, Jeffrey Wu, Clemens Winter, Chris Hesse, Mark Chen, Eric Sigler, Mateusz Litwin, Scott Gray, Benjamin Chess, Jack Clark, Christopher Berner, Sam McCandlish, Alec Radford, Ilya Sutskever, and Dario Amodei. Language models are few-shot learners. In *Advances in Neural Information Processing Systems*, volume 33, pages 1877–1901. Curran Associates, Inc., 2020.

[6] Nicola De Cao, Gautier Izacard, Sebastian Riedel, and Fabio Petroni. Autoregressive entity retrieval. In *International Conference on Learning Representations*, 2021.

[7] Daniel Deutsch, Shyam Upadhyay, and Dan Roth. A general-purpose algorithm for constrained sequential inference. In *Proceedings of the 23rd Conference on Computational Natural Language Learning (CoNLL)*, pages 482–492. Association for Computational Linguistics, 2019.

[8] Darren Edge, Ha Trinh, Newman Cheng, Joshua Bradley, Alex Chao, Apurva Mody, Steven Truitt, and Jonathan Larson. From local to global: A graph RAG approach to query-focused summarization. *arXiv preprint arXiv:2404.16130*, 2024.

[9] Saibo Geng, Martin Josifoski, Maxime Peyrard, and Robert West. Grammar-constrained decoding for structured NLP tasks without finetuning. In *The 2023 Conference on Empirical*

Methods in Natural Language Processing, 2023. URL https://openreview.net/forum?id=KkHY1WGDII.

[10] Gaole He, Yunshi Lan, Jing Jiang, Wayne Xin Zhao, and Ji-Rong Wen. Improving multi-hop knowledge base question answering by learning intermediate supervision signals. In *Proceedings of the 14th ACM International Conference on Web Search and Data Mining (WSDM)*, pages 553–561. ACM, 2021.

[11] Chris Hokamp and Qun Liu. Lexically constrained decoding for sequence generation using grid beam search. In *Proceedings of the 55th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 1535–1546. Association for Computational Linguistics, 2017. doi: 10.18653/v1/p17-1141.

[12] Lei Huang, Weijiang Yu, Weitao Ma, Weihong Zhong, Zhangyin Feng, Haotian Wang, Qianglong Chen, Weihua Peng, Xiaocheng Feng, Bing Qin, and Ting Liu. A survey on hallucination in large language models: Principles, taxonomy, challenges, and open questions. *arXiv preprint arXiv:2311.05232*, 2023.

[13] Ziwei Ji, Nayeon Lee, Rita Frieske, Tiezheng Yu, Dan Su, Yan Xu, Etsuko Ishii, Ye Jin Bang, Andrea Madotto, and Pascale Fung. Survey of hallucination in natural language generation. *ACM Comput. Surv.*, 55(12), March 2023. ISSN 0360-0300. doi: 10.1145/3571730.

[14] Jinhao Jiang, Kun Zhou, Zican Dong, Keming Ye, Xin Zhao, and Ji-Rong Wen. Structgpt: A general framework for large language model to reason over structured data. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pages 9237–9251. Association for Computational Linguistics, 2023. doi: 10.18653/v1/2023.emnlp-main.574.

[15] Zhengbao Jiang, Frank Xu, Luyu Gao, Zhiqing Sun, Qian Liu, Jane Dwivedi-Yu, Yiming Yang, Jamie Callan, and Graham Neubig. Active retrieval augmented generation. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pages 7969–7992. Association for Computational Linguistics, 2023. doi: 10.18653/v1/2023.emnlp-main.495.

[16] Bowen Jin, Gang Liu, Chi Han, Meng Jiang, Heng Ji, and Jiawei Han. Graph chain-of-thought: Augmenting large language models by reasoning on graphs. In *Findings of the Association for Computational Linguistics: ACL 2024*, pages 163–184, 2024.

[17] Nikhil Kandpal, Haikang Deng, Adam Roberts, Eric Wallace, and Colin Raffel. Large language models struggle to learn long-tail knowledge. In *Proceedings of the 40th International Conference on Machine Learning (ICML)*, pages 15696–15707. PMLR, 2023.

[18] Yunshi Lan, Gaole He, Jinhao Jiang, Jing Jiang, Wayne Xin Zhao, and Ji-Rong Wen. Complex knowledge base question answering: A survey. *arXiv preprint arXiv:2108.06688*, 2022.

[19] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. Retrieval-augmented generation for knowledge-intensive NLP tasks. In *Advances in Neural Information Processing Systems*, volume 33, pages 9459–9474. Curran Associates, Inc., 2020.

[20] Kun Li, Tianhua Zhang, Xixin Wu, Hongyin Luo, James R. Glass, and Helen M. Meng. Decoding on graphs: Faithful and sound reasoning on knowledge graphs through generation of well-formed chains. In *Proceedings of the 63rd Annual Meeting of the Associa-*

tion for Computational Linguistics (Volume 1: Long Papers), pages 24349–24364. Association for Computational Linguistics, 2025. doi: 10.18653/v1/2025.acl-long.1186. URL http://dx.doi.org/10.18653/v1/2025.acl-long.1186.

[21] Stephanie Lin, Jacob Hilton, and Owain Evans. TruthfulQA: Measuring how models mimic human falsehoods. In *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics*, pages 3214–3252. Association for Computational Linguistics, 2022.

[22] Ximing Lu, Peter West, Rowan Zellers, Ronan Le Bras, Chandra Bhagavatula, and Yejin Choi. NeuroLogic decoding: (un)supervised neural text generation with predicate logic constraints. In *Proceedings of the 2021 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, pages 4288–4299. Association for Computational Linguistics, 2021.

[23] Linhao Luo, Yuan-Fang Li, Gholamreza Haffari, and Shirui Pan. Reasoning on graphs: Faithful and interpretable large language model reasoning. In *International Conference on Learning Representations*, 2024.

[24] Linhao Luo, Zicheng Zhao, Gholamreza Haffari, Yuan-Fang Li, Chen Gong, and Shirui Pan. Graph-constrained reasoning: Faithful reasoning on knowledge graphs with large language models. In *Proceedings of the 42nd International Conference on Machine Learning*, volume 267 of *Proceedings of Machine Learning Research*, pages 41540–41565. PMLR, 2025. URL https://proceedings.mlr.press/v267/luo25t.html.

[25] Alex Mallen, Akari Asai, Victor Zhong, Rajarshi Das, Daniel Khashabi, and Hannaneh Hajishirzi. When not to trust language models: Investigating effectiveness of parametric and non-parametric memories. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics*, pages 9802–9822. Association for Computational Linguistics, 2023.

[26] Costas Mavromatis and George Karypis. Gnn-rag: Graph neural retrieval for efficient large language model reasoning on knowledge graphs. In *Findings of the Association for Computational Linguistics: ACL 2025*, pages 16682–16699. Association for Computational Linguistics, 2025. doi: 10.18653/v1/2025.findings-acl.856.

[27] Ethan Perez, Sam Ringer, Karin=ě Lukoši=ťe, Katy Nguyen, Edwin Chen, Scott Heiner, Craig Pettit, Catherine Olsson, Sandipan Kundu, Saurav Kadavath, Andy Jones, Anna Chen, Ben Mann, Brian Israel, Samuel R. Bowman, Jack Clark, Deep Ganguli, Amanda Askell, and Danny Hernandez. Discovering language model behaviors with model-written evaluations. In *Findings of the Association for Computational Linguistics: ACL 2023*, pages 13387–13434. Association for Computational Linguistics, 2023.

[28] Gabriel Poesia, Oleksandr Polozov, Vu Le, Ashish Tiwari, Gustavo Soares, Christopher Meek, and Sumit Gulwani. Synchromesh: Reliable code generation from pre-trained language models. In *International Conference on Learning Representations*, 2022.

[29] Nils Reimers and Iryna Gurevych. Sentence-BERT: Sentence embeddings using siamese BERT-networks. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 3982–3992. Association for Computational Linguistics, 2019.

[30] Apoorv Saxena, Soumen Chakrabarti, and Partha Talukdar. Sequence-to-sequence knowledge graph completion and question answering. In *Proceedings of the 60th Annual Meeting of the*

Association for Computational Linguistics, pages 2814–2828. Association for Computational Linguistics, 2022.

[31] Torsten Scholak, Nathan Schucher, and Dzmitry Bahdanau. Picard: Parsing incrementally for constrained auto-regressive decoding from language models. In *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing*. Association for Computational Linguistics, 2021.

[32] Jiabin Shi, Shulin Cao, Lei Hou, Juanzi Li, and Hanwang Zhang. transfernet: An effective and transparent framework for multi-hop question answering over relation graph. In *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing*, pages 4149–4158. Association for Computational Linguistics, 2021.

[33] Jiashuo Sun, Chengjin Xu, Lumingyuan Tang, Saizhuo Wang, Chen Lin, Yeyun Gong, Lionel M. Ni, Heung-Yeung Shum, and Jian Guo. Think-on-graph: Deep and responsible reasoning of large language model on knowledge graph. In *International Conference on Learning Representations*, 2024.

[34] Alon Talmor and Jonathan Berant. The web as a knowledge-base for answering complex questions. In *Proceedings of the 2018 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (NAACL-HLT)*, pages 641–651. Association for Computational Linguistics, 2018.

[35] Harsh Trivedi, Niranjan Balasubramanian, Tushar Khot, and Ashish Sabharwal. Interleaving retrieval with chain-of-thought reasoning for knowledge-intensive multi-step questions. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics*, pages 10014–10037. Association for Computational Linguistics, 2023.

[36] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems*, 2017.

[37] Xuezhi Wang, Jason Wei, Dale Schuurmans, Quoc V Le, Ed H. Chi, Sharan Narang, Aakanksha Chowdhery, and Denny Zhou. Self-consistency improves chain of thought reasoning in language models. In *International Conference on Learning Representations*, 2023.

[38] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Brian Ichter, Fei Xia, Ed Chi, Quoc Le, and Denny Zhou. Chain-of-thought prompting elicits reasoning in large language models. In *Advances in Neural Information Processing Systems*, volume 35, pages 24824–24837, 2022.

[39] Brandon T. Willard and Rémi Louf. Efficient guided generation for large language models, 2023.

[40] Ziwei Xu, Sanjay Jain, and Mohan Kankanhalli. Hallucination is inevitable: An innate limitation of large language models, 2025.

[41] Shunyu Yao, Dian Yu, Jeffrey Zhao, Izhak Shafran, Tom Griffiths, Yuan Cao, and Karthik Narasimhan. Tree of thoughts: Deliberate problem solving with large language models. In *Advances in Neural Information Processing Systems*, volume 36, pages 11809–11822, 2023.

[42] Michihiro Yasunaga, Hongyu Ren, Antoine Bosselut, Percy Liang, and Jure Leskovec. QA-GNN: Reasoning with language models and knowledge graphs for question answering. In *Proceedings of the 2021 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, pages 535–546. Association for Computational Linguistics, 2021.

[43] Susumu Yata et al. marisa-trie: Matching algorithm with recursively implemented storage. `https://github.com/s-yata/marisa-trie`, 2024.

[44] Wen-tau Yih, Matthew Richardson, Christopher Meek, Ming-Wei Chang, and Jina Suh. The value of semantic parse labeling for knowledge base question answering. In *Proceedings of the 54th Annual Meeting of the Association for Computational Linguistics*, pages 201–206. Association for Computational Linguistics, 2016.

[45] Donghan Yu, Chenguang Chen, Yin Onoe, and Peng Qi. Decaf: Joint decoding of answers and logical forms for question answering over knowledge bases. In *International Conference on Learning Representations*, 2023.

[46] Bo Yuan, Hexuan Deng, Xuebo Liu, and Min Zhang. Routerkgqa: Specialized–general model routing for constraint-aware knowledge graph question answering, 2026.

[47] Honghua Zhang, Meihua Dang, Nanyun Peng, and Guy Van den Broeck. Tractable control for autoregressive language generation. In *International Conference on Machine Learning*, 2023.

[48] Alex Zhuang, Ge Zhang, Tianyu Zheng, Xinrun Du, Junjie Wang, Weiming Ren, Wenhao Huang, Jie Fu, Xiang Yue, and Wenhu Chen. StructLM: Towards building generalist models for structured knowledge grounding. In *First Conference on Language Modeling*, 2024.

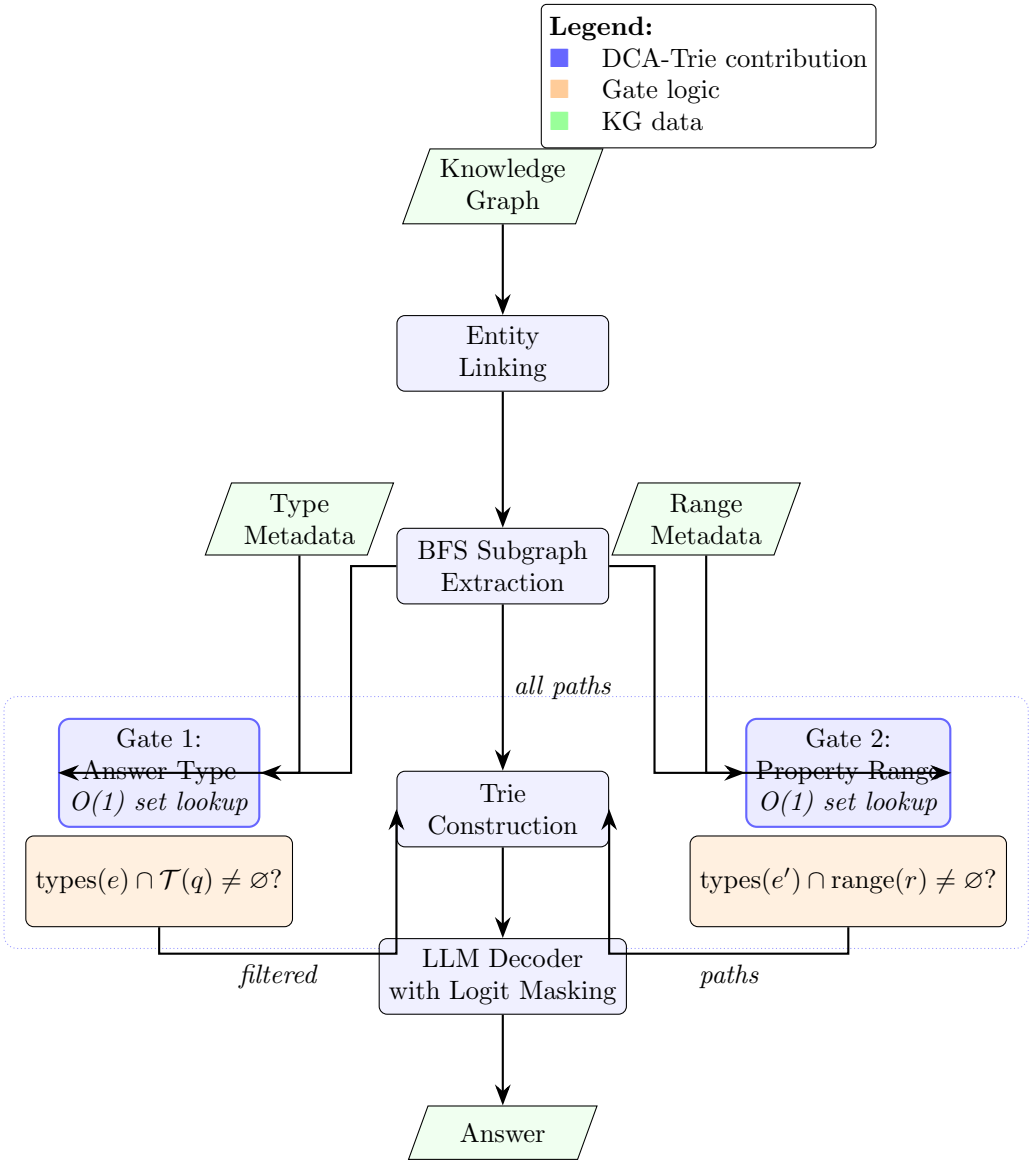

Figure 5: DCA-Trie system architecture. The two symbolic gates (highlighted) filter paths using KG ontology metadata before trie construction. Both gates are $O(1)$ set lookups, require no encoder or GPU, and default to conservative (pass) when metadata is absent.